School of Computer Science and Engineering

JAIN (DEEMED-TO-BE) UNIVERSITY

B. Tech CSE-AIML

PROJECT REPORT ON

"Spam Message Detection"

Submitted in partial fulfilment of the requirements

BACHELOR OF TECHNOLOGY IN

COMPUTER SCIENCE AND AIML

Submitted by

SNO	NAME	USN
1.	D.Akhil	23BTRCL256

Under the guidance of

Dr. Swathi Gupta

Assistant Professor

DECLARATION

I, D. Akhil, bearing USN 23BTRCL256, hereby solemnly declare that the project work entitled **"Spam Message Detection in Social Networks"** is a Bonafide record of the work carried out by me as part of my academic requirements. This project has been completed under the valuable guidance and supervision of **Dr. Swathi Gupta, Associate Professor, Department of Computer Science and Engineering (Artificial Intelligence & Machine Learning), JAIN (Deemed-to-be University)**.

I affirm that this project represents my own original work and effort. The work presented in this report has been carried out by me independently, and all sources of information used in this study have been appropriately acknowledged and cited wherever necessary. I have taken due care to ensure the authenticity, accuracy, and integrity of the data and results included in this project.

I further declare that this project work has not been previously submitted, either in full or in part, to any other university, institution, or organization for the award of any degree, diploma, or certification. The work submitted here is entirely original and complies with the academic guidelines and ethical standards prescribed by the institution.

I understand that any form of plagiarism or academic dishonesty, if detected at any stage, may lead to disciplinary action as per the rules and regulations of the university.

Signature of the Student
D. Akhil
USN: 23BTRCL256

CERTIFICATE

This is to certify that the project entitled "**Spam Message Detection in Social Networks**" is a bonafide record of the work done by D.Akhil (23BTRCL256) of the B.Tech (Artificial Intelligence and Machine Learning) programme, Jain University, during the academic year 2023–2027.

The project has been carried out under my guidance and supervision.
The work presented is original and has not been submitted, either in part or in full, for the award of any other degree or

diploma at this or any other institution.

Dr. V Chandrashekar
Computer Science and Engineering,
School of Computer Science & Engineering, Faculty of Engineering & Technology
JAIN (Deemed to be University)

Dr. Swathi Gupta

Computer Science and Engineering,

Department of AIML
Jain University

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who have contributed to the successful completion of this project.

First and foremost, I extend my heartfelt thanks to my project guide, ***Dr. Swathi Gupta****, Associate Professor,* ***Dr. Chandra Sekhar*** *Department of CSE-AIML, for his invaluable guidance, continuous support, and encouragement throughout this project. His expertise in the field of Artificial Intelligence and Machine Learning has been instrumental in shaping this work.*

I am grateful to the Head of the Department and all the faculty members of the Department of CSE-AIML for providing the necessary facilities and a conducive environment for carrying out this project.

I also thank the management of JAIN (Deemed-to-be University) for providing excellent infrastructure and resources that enabled the successful completion of this work.

I extend my appreciation to my classmates and friends for their constant motivation and support during the course of this project.

Finally, I am deeply indebted to my parents and family for their unwavering support, patience, and encouragement throughout my academic journey.

D. Akhil

Signature of Student

TABLE OF CONTENTS

Chapter No.	Title	Page No.
	Certificate	i
	Declaration	ii
	Acknowledgement	iii
	Abstract	iv
	List of Figures	v
	List of Tables	vi
1	Introduction	10-12
1.1	Background and Motivation	10
1.2	Problem Statement	10-11
1.3	Scope of the Project	11
1.4	Objectives of the Project	11-12
2	Literature Survey	13-16
2.1	Resume Parsing and Information Extraction	13
2.2	Job Recommendation Systems	13-14
2.3	Candidate Ranking and Relevance Assessment	14-15
2.4	Natural Language Processing for Recruitment	15-16

2.5	Research Gap and Contribution	16
3	Existing System	18-20
3.1	Traditional Job Portal Approaches	18
3.2	Keyword Matching Limitations	18-19
3.3	Limitations of Existing Approaches	19-20
4	Proposed System	21-26
4.1	System Overview	21
4.2	System Architecture	21
4.3	Data Flow	22-23
4.4	AI Pipeline Architecture	23-24
4.5	Compatibility Scoring Mechanism	24
4.6	Ranking Mechanism	25
4.7	Recommendation Engine	25-26
5	System Modules	27-31
5.1	Resume Upload Module	27
5.2	Text Preprocessing Module	28
5.3	Skill Extraction Module	28
5.4	Matching and Scoring Module	29
5.5	Ranking Module	29
5.6	Recommendation Engine Module	30

5.7	Frontend Module	31
5.8	Backend Module	31
6	Implementation	32-34

6.1	Tools and Technologies	32
6.2	MERN Stack Implementation	32-33
6.3	Dataset Preparation	33
6.4	Model Development	33
6.5	API Development	33-34
6.6	Frontend Development	34
6.7	Integration and Deployment	34
7	Results and Analysis	35-40
7.1	Experimental Setup	35
7.2	Performance Metrics	35
7.3	Results Explanation	36
7.4	Comparison with Existing Systems	38-39
7.5	Discussion	39-40
8	Publications	38-39
8.1	Published Paper (Zenodo)	41
8.2	Submitted Paper (Conference / IEEE)	42
9	Conclusion and Future Work	43-46

9.1	Conclusion	43
9.2	Achievements	43-44
9.3	Limitations	44
9.4	Future Enhancements	44-46
	References	47-48

LIST OF TABLES

Table No.	Title	Page No.
Table 1	Literature Survey Comparison	17
Table 2	Comparison with Existing Systems	39

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 1	System Architecture of Smart Hire	22
Figure 2	DataFlow Diagram	23
Figure 3	Matching Flowchart	25
Figure 4	Resume Processing Workflow	27
Figure 5	Candidate Ranking Process	30
Figure 6	Home Page Interface	36
Figure 7	Resume Upload and Skill Extraction Output	36

Figure 8	**Job Recommendation**	**37**
Figure 9	**Skill Gap Analysis**	**37**
Figure 10	**Employer Dashboard**	**38**
Figure 11	**Candidate Ranking Outputs**	**38**

ABSTRACT

Spam messages have become a significant challenge in modern digital communication systems, particularly across social networks, messaging platforms, and email services. These unwanted messages not only reduce user experience but also pose serious risks such as phishing attacks, fraud, and the spread of malicious content. This project presents the design and implementation of a **Spam Message Detection System**, an AI-based solution that automates the identification and classification of spam messages using Natural Language Processing (NLP) techniques.

The system processes textual data from messages using preprocessing techniques such as tokenization, stopword removal, and lemmatization to clean and standardize the input. Feature extraction methods are applied to convert text into meaningful representations suitable for classification. The system then utilizes machine learning algorithms such as Naive Bayes and Logistic Regression to classify messages as spam or non-spam (ham). Similarity-based techniques and vectorization methods such as TF-IDF were also explored to improve classification accuracy.

The model was trained and tested on standard datasets consisting of labeled spam and non-spam messages collected from real-world communication platforms. The results demonstrate that the system effectively identifies spam messages with high accuracy and significantly improves detection compared to traditional keyword-based filtering methods.

The system is implemented as a full-stack web application using the MERN (MongoDB, Express, React, Node.js) technology stack. The frontend provides a user-friendly interface for message input and result display, while the backend handles preprocessing, feature extraction, and classification through RESTful APIs. MongoDB is used for efficient data storage and management.

This project demonstrates that NLP and machine learning techniques can enhance spam detection systems by providing scalable, efficient, and intelligent message filtering, thereby improving user safety and communication quality.

Keywords: Spam Detection, Natural Language Processing, Text Classification, Machine Learning, TF-IDF, Naive Bayes, MERN Stack.

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

The rapid growth of digital communication platforms such as social networks, messaging applications, and email services has significantly transformed the way people communicate. However, this transformation has also led to a dramatic increase in unwanted and harmful messages, commonly known as spam. Spam messages include advertisements, phishing links, fraudulent schemes, and malicious content that negatively impact user experience and pose security risks.

With the increasing volume of online communication, manually identifying and filtering spam messages has become impractical and inefficient. Traditional filtering methods, which rely on simple keyword-based rules, often fail to accurately detect spam due to variations in language, use of abbreviations, and evolving spam techniques. This results in important messages being misclassified or spam messages bypassing filters.

Recent advancements in Artificial Intelligence (AI) and Natural Language Processing (NLP) provide powerful tools to automatically analyze and classify textual data. NLP techniques enable systems to understand and process unstructured text, making it possible to detect patterns and features that distinguish spam from legitimate messages.

Motivated by these challenges, this project focuses on developing an intelligent Spam Message Detection System that leverages NLP and machine learning techniques to automatically classify messages as spam or non-spam, thereby improving communication security and user experience.

1.2 Problem Statement

Despite the availability of spam filtering mechanisms, several challenges still exist in effectively detecting spam messages:

• Inefficient Manual Filtering: Manual identification of spam messages is time-consuming and not scalable for large volumes of data.

• Keyword-Based Limitations: Traditional systems rely on keyword matching,

which fails to detect variations, synonyms, and contextual meanings in messages.
• Evolving Spam Techniques: Spammers continuously modify message patterns to bypass existing filters.
• False Positives and Negatives: Legitimate messages may be incorrectly classified as spam, while some spam messages may go undetected.
• Scalability Issues: Existing systems struggle to efficiently process large-scale real-time data.
These challenges highlight the need for an automated and intelligent system capable of accurately detecting spam messages while adapting to changing patterns.

1.3 Scope of the Project

The scope of the Spam Message Detection System includes:
• Text Preprocessing: Cleaning and preparing message data using techniques such as tokenization, stopword removal, and lemmatization.
• Feature Extraction: Converting text into numerical representations using methods such as TF-IDF or Bag-of-Words.
• Spam Classification: Applying machine learning algorithms such as Naive Bayes or Logistic Regression to classify messages.
• Performance Evaluation: Measuring model accuracy using metrics such as precision, recall, and F1-score.
• Full-Stack Implementation: Developing a web-based application using the MERN stack for user interaction and real-time prediction.
The project does not include advanced deep learning models or large-scale real-time deployment, which are considered future enhancements.

1.4 Objectives of the Project

The main objectives of this project are:
• To develop a system that preprocesses and analyzes message data using NLP techniques.
• To implement feature extraction methods for effective text representation.

- To build a machine learning model for accurate spam message classification.
- To evaluate the system using standard performance metrics.
- To develop a user-friendly web application using the MERN stack.
- To demonstrate how AI and NLP techniques can improve spam detection and reduce manual effort.

The project does not include advanced features such as multilingual detection, real-time streaming analysis, or deep learning-based classification, which are considered potential future enhancements.

CHAPTER 2: LITERATURE SURVEY

2.1 Spam Message Detection Techniques

Spam message detection is a critical area in text classification and cybersecurity, focusing on identifying unwanted or harmful messages from large volumes of communication data. Early approaches to spam detection relied on rule-based filtering systems, which used predefined patterns, blacklists, and keyword matching techniques to identify spam messages. While these methods were simple and easy to implement, they lacked adaptability and failed to generalize across evolving spam patterns.

Modern spam detection systems leverage machine learning techniques to automatically learn patterns from labelled datasets. These approaches treat spam detection as a binary classification problem, where messages are classified as spam or non-spam (ham). Common algorithms such as Naive Bayes, Support Vector Machines (SVM), and Decision Trees have been widely used due to their effectiveness and efficiency. These models learn from historical data and can generalize better than rule-based systems.

However, spam detection remains challenging due to variations in language, obfuscation techniques used by spammers, and the dynamic nature of spam content. Messages may include misspellings, symbols, or intentional distortions to bypass detection systems, making it difficult to maintain high accuracy.

2.2 Machine Learning-Based Spam Filtering

Machine learning–based spam filtering is an advanced technique used to automatically classify messages (such as emails or SMS) as **spam (unwanted)** or **ham (legitimate)** by learning patterns from data instead of relying on fixed rules. Unlike traditional filtering methods, machine learning models improve their accuracy over time by analysing large datasets of labelled messages.

1. Working Principle

The core idea behind machine learning spam filtering is to train a model using previously labelled messages. These messages are categorized as spam or ham,

and the model learns distinguishing patterns such as commonly used words, phrases, and structures found in spam messages.
Once trained, the model can predict the category of new incoming messages automatically.

2. Steps Involved in Spam Filtering

a) Data Collection

A dataset is collected containing labelled messages:

- **Spam messages** (e.g., promotional offers, fake prizes)
- **Ham messages** (normal conversations)

Example datasets include SMS Spam Collection datasets.

b) Text Preprocessing

Raw text cannot be directly used by machine learning models, so it is cleaned and prepared using the following steps:

- **Lowercasing** – Convert all text to lowercase
- **Tokenization** – Split sentences into words
- **Stop word Removal** – Remove common words like "is", "the", "and"
- **Stemming/Lemmatization** – Reduce words to root form (e.g., "winning" → "win")
- **Removing punctuation and special characters**

This step ensures that only meaningful information is retained.

c) Feature Extraction

Machine learning models require numerical input, so text is converted into numbers using techniques such as:

- **Bag of Words (BoW):** Counts word frequency
- **TF-IDF (Term Frequency–Inverse Document Frequency):** Measures importance of words
- **N-grams:** Captures sequences of words

TF-IDF is widely used because it gives higher importance to rare but meaningful words (e.g., "prize", "winner").

3. Machine Learning Algorithms Used

Several algorithms are commonly used for spam detection:

a) Naïve Bayes

- Most popular for text classification
- Based on probability and Bayes' theorem
- Fast and efficient for large datasets

b) Logistic Regression

- Used for binary classification
- Predicts probability of spam or ham

c) Support Vector Machine (SVM)

- Finds optimal boundary between spam and ham
- Works well with high-dimensional data

d) Decision Trees & Random Forest

- Tree-based models for classification
- Handle complex patterns effectively

Among these, **Naïve Bayes** is widely preferred due to its simplicity and strong performance in text classification tasks.

4. Model Training and Testing

The dataset is split into:

- **Training Data (e.g., 80%)** – Used to train the model
- **Testing Data (e.g., 20%)** – Used to evaluate performance

The model learns patterns during training and is then tested on unseen data to measure accuracy.

5. Evaluation Metrics

To measure performance, the following metrics are used:

- **Accuracy:** Overall correctness
- **Precision:** How many predicted spam messages are actually spam
- **Recall:** How many actual spam messages are correctly identified
- **F1-Score:** Balance between precision and recall

These metrics help ensure the model is reliable and not biased.

2.3 Text Representation and Feature Extraction

Feature extraction plays a crucial role in spam detection systems. The effectiveness of classification largely depends on how well textual data is represented.

Traditional methods such as Bag-of-Words represent text as a collection of word frequencies, ignoring word order but capturing term occurrence. TF-IDF improves upon this by assigning higher weights to important words and reducing the impact of common words.

More advanced approaches include word embeddings such as Word2Vec, GloVe, and FastText, which represent words as dense vectors in continuous space. These embeddings capture semantic relationships between words, allowing models to understand context and similarity.

Feature engineering techniques such as n-grams (sequences of words), character-level features, and frequency-based filtering are also used to improve detection accuracy. However, selecting the right features remains a challenge and often requires domain knowledge.

2.4 Natural Language Processing for Spam Detection

Natural Language Processing (NLP) provides essential tools for analyzing and processing unstructured text in spam detection systems. Text preprocessing is the first step in the NLP pipeline and includes tokenization, stopword removal, case normalization, and lemmatization. These steps help clean and standardize the data, improving model performance.

Advanced NLP techniques such as Named Entity Recognition (NER) and part-of-speech tagging can provide deeper insights into message structure, although they are not always necessary for basic spam detection tasks.

Deep learning models such as Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM) networks, and transformer-based models like BERT have shown significant improvements in text classification tasks. These models capture contextual information and dependencies between words, leading to higher accuracy in detecting complex spam patterns.

However, deep learning approaches require large datasets and high computational resources, which may not be feasible for all applications.

2.5 Research Gap and Contribution

Although significant research has been conducted in spam detection, several challenges remain unresolved. Many existing systems focus on either rule-based or machine learning approaches but do not provide a fully integrated, user-friendly solution. Additionally, traditional models often struggle with evolving spam patterns, semantic understanding, and real-time classification.

Another major limitation is the trade-off between model complexity and interpretability. While advanced models such as deep learning provide high accuracy, they often act as black-box systems, making it difficult to understand their decision-making process.

This project addresses these gaps by developing a practical and efficient Spam Message Detection System that combines NLP preprocessing techniques with machine learning-based classification. The system focuses on simplicity, interpretability, and real-time usability by implementing effective text preprocessing, feature extraction, and classification methods within a full-stack web application.

The key contributions of this project include:

1. Implementation of an end-to-end spam detection pipeline from data preprocessing to classification.
2. Use of NLP techniques for effective text cleaning and feature extraction.
3. Application of machine learning algorithms for accurate spam classification.
4. Development of a user-friendly web-based interface using the MERN stack.
5. Evaluation of system performance using standard metrics such as accuracy, precision, and recall.

Table 1: Literature Survey

Sr. No.	Paper Title	Author(s)	Year	Methods
1	Neural Architectures for Named Entity Recognition	Lample et al.	2016	BiLSTM, CRF
2	BERT: Pre-training of Deep Bidirectional Transformers	Devlin et al.	2019	Transformer, NLP
3	Scikit-learn: Machine Learning in Python	Pedregosa et al.	2011	ML Library
4	Research-Paper Recommender Systems: A Literature Survey	Beel et al.	2016	Recommendation Systems
5	SMS Spam Filtering using Machine Learning Techniques	Almeida et al.	2011	Naive Bayes, NLP
6	Learning to Rank for Information Retrieval	Liu, T.Y.	2009	Ranking Algorithms

CHAPTER 3: EXISTING SYSTEM

3.1 Traditional Spam Detection Approaches

Existing spam detection systems are widely used in email services, messaging platforms, and social networks to filter unwanted messages. Traditional systems primarily rely on rule-based filtering techniques, where predefined rules, blacklists, and keyword matching are used to classify messages as spam or non-spam.

These systems function by scanning messages for specific words, phrases, or patterns commonly associated with spam content, such as promotional terms, suspicious links, or repeated characters. Some systems also use sender-based filtering, where messages from unknown or blacklisted sources are automatically marked as spam.

While these approaches provide a basic level of protection, they place a significant burden on predefined rules and manual updates. As spam techniques evolve rapidly, maintaining and updating these rules becomes challenging. Additionally, users often need to manually review filtered messages, leading to inefficiencies.

3.2 Keyword-Based Filtering Limitations

Keyword-based filtering is one of the most commonly used approaches in traditional spam detection systems. It operates by identifying specific words or patterns within messages and classifying them accordingly. However, this approach has several limitations:

• Synonym and Language Variation: Spam messages often use alternative words, abbreviations, or misspellings to bypass filters. For example, “free” may be written as “fr33” or “f.re.e,” making detection difficult.

• Context Insensitivity: Keyword-based systems do not understand the context of words. A legitimate message containing promotional words may be incorrectly classified as spam, while cleverly disguised spam may pass through.

• Obfuscation Techniques: Spammers intentionally manipulate text using symbols, spacing, or encoding to evade detection, which keyword-based systems struggle to handle.
• High False Positives and Negatives: Important messages may be marked as spam (false positives), while actual spam messages may not be detected (false negatives).
• Scalability Issues: As the volume of messages increases, keyword-based systems require continuous updates and become less efficient in handling large-scale data.

3.3 Limitations of Existing Approaches

Beyond keyword-based filtering, traditional spam detection systems face several broader challenges:
• Lack of Adaptability: Rule-based systems cannot automatically adapt to new and evolving spam patterns, requiring frequent manual updates.
• Limited Learning Capability: Traditional systems do not learn from past data, making them less effective in detecting new types of spam.
• Manual Intervention: Users often need to manually mark messages as spam or not spam, which is time-consuming and inefficient.
• Poor Accuracy: Due to limited understanding of language and context, existing systems may produce inaccurate results.
• Inefficient Real-Time Processing: Handling large volumes of messages in real-time can be challenging for traditional systems.
• Security Risks: Ineffective spam detection can allow phishing messages and malicious links to reach users, posing serious security threats.

These limitations highlight the need for an advanced spam detection system that leverages Artificial Intelligence (AI) and Natural Language Processing (NLP) techniques to automatically learn patterns, improve accuracy, and efficiently handle large-scale message data.

CHAPTER 4: PROPOSED SYSTEM

4.1 System Overview

The **Spam Message Detection System** is an intelligent automated text classification system developed to distinguish between spam (unwanted or malicious messages) and ham (legitimate messages). The system leverages **Natural Language Processing (NLP)** and **Machine Learning (ML)** techniques to analyze textual data and make accurate predictions.

In modern communication platforms such as email, messaging applications, and social media, spam messages pose significant challenges including phishing attacks, fraud, and unnecessary clutter. This system addresses these challenges by providing an automated filtering mechanism that reduces human intervention and enhances communication safety.

The proposed system is designed with the following objectives:

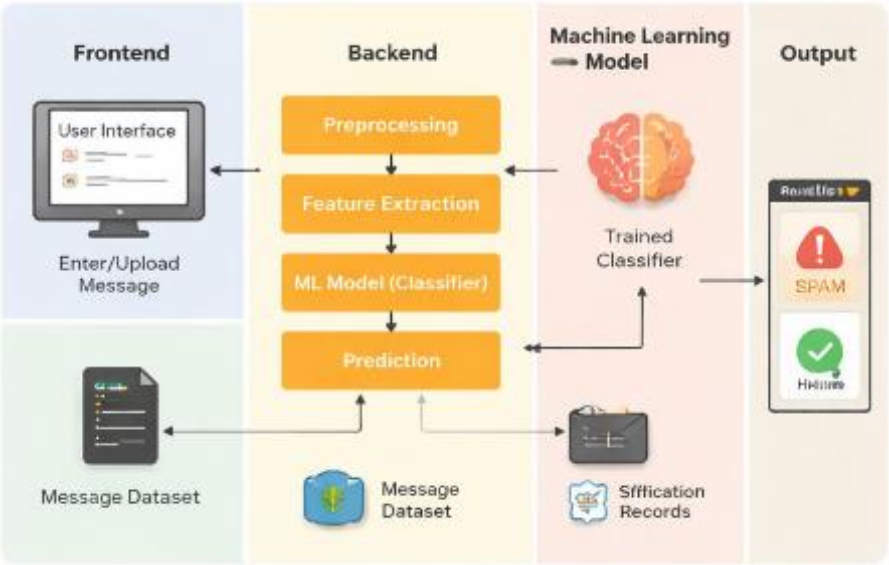
- To automatically classify incoming messages as spam or ham
- To reduce manual effort in filtering messages
- To improve detection accuracy using machine learning models
- To ensure fast and real-time prediction

The system processes raw textual input, extracts meaningful patterns, and applies a trained classification model to generate predictions. It is scalable and can be integrated into real-world applications such as email services, SMS filtering systems, and chat applications.

4.2 System Architecture

The diagram represents the overall system architecture, which is divided into multiple layers including the frontend, backend, machine learning model, and output layer. The frontend provides a user interface for entering or uploading messages. The backend handles the core processing tasks such as preprocessing, feature extraction, and classification. The

machine learning model, trained on a labeled dataset, performs the actual prediction based on extracted features. The data components include the message dataset used for training and classification records for storing results. Finally, the output layer displays the prediction result to the user in the form of spam or ham classification. Together, these components ensure a structured, efficient, and scalable system for real-time spam detection.



4.3 Data Flow

The diagrams illustrate the working and architecture of the Spam Message Detection System. The first diagram represents the processing workflow of the system, where an input message provided by the user is initially passed through a text preprocessing stage. In this stage, the message is cleaned by removing punctuation, converting text to lowercase, and tokenizing it into meaningful words. The cleaned text is then transformed into numerical form using feature extraction techniques such as TF-IDF, which assigns importance to each word based on its relevance. These features are then fed into a trained machine learning classifier, such as Naïve Bayes, Logistic Regression, or Support Vector Machine, which analyzes the patterns learned during training and predicts whether the

message is spam or ham. The final output is displayed to the user, indicating whether the message is safe or potentially harmful.

Figure 2: Data Flow Diagram

4.4 AI Pipeline Architecture

The AI pipeline transforms raw text into structured input suitable for machine learning models.

1. Text Preprocessing

Text preprocessing improves data quality and ensures consistency.

Steps Involved:

- **Lowercasing**: Converts all text to lowercase for uniformity
- **Punctuation Removal**: Eliminates symbols such as (!, ?, ., etc.)
- **Stop Word Removal**: Removes common words (e.g., "the", "is", "and")
- **Tokenization**: Splits text into individual words or tokens
- **Stemming/Lemmatization**: Reduces words to root form

Example:

Input:

"Congratulations! You have WON a prize!!!"

Output:

"Congratulation win prize"

2. Feature Extraction (TF-IDF)

TF-IDF (Term Frequency-Inverse Document Frequency) converts text into numerical vectors.

- **Term Frequency (TF)**: Frequency of a word in a message
- **Inverse Document Frequency (IDF)**: Importance of the word across dataset

Advantages:

- Highlights important words
- Reduces impact of common words
- Improves model accuracy

3. Classification Models

The system supports multiple machine learning models:

Naïve Bayes

- Based on probability
- Fast and efficient for text classification

Logistic Regression

- Linear model for binary classification
- Provides probability output

Support Vector Machine (SVM)

- Effective for high-dimensional data
- Maximizes classification margin

Pipeline Summary

Raw Text → Preprocessing → TF-IDF → Model → Prediction

4.5 Spam Classification Mechanism

The classification mechanism uses learned patterns from labeled data.

Working Principle:

- Extract features using TF-IDF
- Feed features into trained classifier
- Compute probability score

Decision Rule:

- If Probability ≥ Threshold → Spam
- Else → Ham

Threshold Value:

Typically set to 0.5 but can be tuned for better performance.

Model Training Process:

- Dataset split into training and testing sets
- Model trained on labelled data
- Performance evaluated using:
  - Accuracy
  - Precision

- Recall
- F1-Score

Advanced Techniques (Future Scope):

- **LSTM (Long Short-Term Memory)** for sequence learning
- **BERT (Bidirectional Encoder Representations from Transformers)** for contextual understanding

These models improve performance but require higher computational resources.

4.6 Decision Mechanism

The decision mechanism ensures accurate classification.

Step-by-Step Process:

1. Accept input message
2. Perform preprocessing
3. Convert to TF-IDF features
4. Apply trained model
5. Generate prediction
6. Display result

Decision Characteristics:

- Fast execution
- Deterministic output
- Scalable for large inputs

4.7 Prediction and Alert System

This module provides real-time feedback to users.

Output Types:

Ham Message

- Marked as safe
- No warning displayed

Spam Message

- Warning alert generated
- Highlighted as suspicious

Alert Features:

- Instant notification
- Visual indicators (color coding)

CHAPTER 5: SYSTEM MODULES

5.1 Message Input Module

The Message Input Module provides the interface through which users can enter or upload messages for spam detection. This module acts as the entry point of the system and is accessible through the user interface. Users can either type a message manually or upload a dataset containing multiple messages.

The module validates the input to ensure it is not empty and is in a proper format. Basic checks such as text length and allowed characters are performed before processing. Once validated, the input message is securely passed to the backend for further analysis. This module ensures smooth interaction between the user and the system while maintaining input quality.

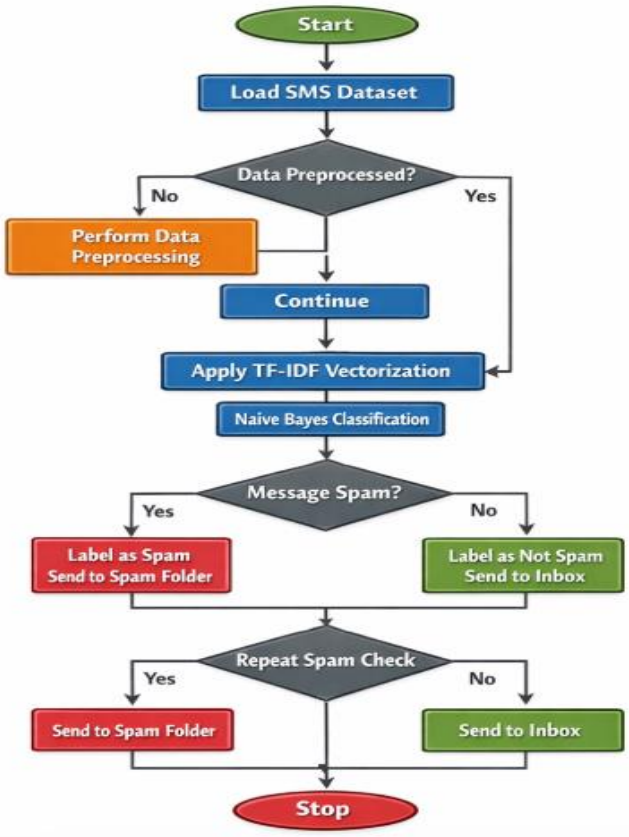


Figure 4: Message Processing Workflow

5.2 Text Preprocessing Module

The Text Preprocessing Module is responsible for cleaning and preparing raw message text for analysis. Since raw text contains noise such as punctuation, special characters, and irrelevant words, preprocessing ensures that only meaningful data is retained.

The module performs several operations, including converting all text to lowercase to maintain uniformity, removing extra spaces and punctuation, and eliminating stop words that do not contribute to classification. Tokenization is applied to split the message into individual words, and stemming or lemmatization is used to reduce words to their base form.

These preprocessing steps significantly improve the quality of the input data and enhance the performance of the machine learning model by reducing redundancy and noise

5.3 Feature Extraction Module

The Feature Extraction Module converts the cleaned text into numerical format so that it can be processed by machine learning algorithms. This is achieved using the TF-IDF (Term Frequency-Inverse Document Frequency) vectorization technique.

TF-IDF assigns weights to each word based on its importance in the message relative to the entire dataset. Words that occur frequently in a message but are rare across the dataset are given higher importance. This helps in distinguishing spam messages, which often contain specific keywords such as “free,” “win,” or “offer.”

The resulting feature vectors represent the textual data in a structured numerical form, making it suitable for classification.

5.4 Classification Module

The Classification Module is the core component of the system that determines whether a message is spam or ham. It uses a trained machine learning model, such as Naïve Bayes, Logistic Regression, or Support Vector Machine, to perform this task.

The module receives TF-IDF feature vectors as input and applies the trained model to analyse patterns learned from historical data. Based on this analysis, the model predicts the category of the message.

Naïve Bayes is commonly used due to its efficiency and effectiveness in text classification problems. The model computes probabilities and assigns the message to the most likely class.

5.5 Decision and Filtering Module

The Decision and Filtering Module generates the final classification result based on the model's prediction. If the message is classified as spam, it is labelled accordingly and can be directed to a spam folder. If classified as ham, it is marked as safe and allowed to remain in the inbox.

The module may also use a threshold value to improve decision accuracy. Messages with higher probability scores are confidently classified as spam, while others are treated as safe. This module ensures that users are protected from unwanted or harmful messages.

5.6 Prediction and Alert Module

The Prediction and Alert Module provide real-time feedback to the user after classification. Once the system determines whether a message is spam or ham, the result is displayed clearly on the interface.

Spam messages trigger warning alerts, often highlighted visually to indicate risk, while ham messages are shown as safe. This immediate feedback helps users make informed decisions and avoid interacting with suspicious content.

The module can be extended in the future to support real-time notifications, automatic filtering, and integration with messaging platforms.

5.7 Frontend Module

The Frontend Module is responsible for the user interface of the system. It allows users to input messages, upload datasets, and view classification results. The interface is designed to be simple, user-friendly, and responsive.

Technologies such as HTML, CSS, JavaScript, or frameworks like React can be used to build the frontend. The module ensures seamless interaction between the user and the system and provides clear visualization of results.

5.8 Backend Module

The Backend Module handles the core processing and communication between different components of the system. It is responsible for executing preprocessing, feature extraction, classification, and result generation.

The backend can be implemented using technologies such as Python (Flask/Django) or Node.js. It also manages the trained machine learning model and processes incoming requests from the frontend.

Additionally, the backend interacts with the data storage system, which contains training datasets, model files, and prediction records. This ensures efficient data handling, system performance, and scalability.

.

CHAPTER 6: IMPLEMENTATION

6.1 Tools and Technologies

The Spam Message Detection System is implemented using a combination of modern technologies that ensure efficient processing, scalability, and accurate prediction of spam messages.

Programming Language:

Python is primarily used for implementing the machine learning model and backend logic due to its simplicity and strong support for data processing and machine learning libraries.

Text Processing:

Natural Language Processing (NLP) techniques are implemented using libraries such as NLTK and Scikit-learn. These are used for tokenization, stopword removal, stemming, and text normalization.

Machine Learning:

Scikit-learn is used to implement classification algorithms such as Naïve Bayes, Logistic Regression, and Support Vector Machine. TF-IDF vectorization is used for feature extraction.

Visualization (Optional):

Libraries such as Matplotlib or Seaborn can be used for performance visualization, including accuracy graphs and confusion matrices.

The system focuses on simplicity and efficiency while ensuring reliable spam detection performance.

6.2 System Implementation Architecture

The system is implemented using a modular architecture consisting of frontend, backend, and machine learning components.

Frontend:

The user interface allows users to input messages and view results. It can be developed using HTML, CSS, and JavaScript or frameworks like React for better interactivity.

Backend:
The backend is implemented using Python frameworks such as Flask or Django. It handles user requests, processes input data, and communicates with the machine learning model.
Machine Learning Model:
The trained model is integrated into the backend and is used to classify incoming messages in real time.
Data Storage:
Datasets and trained models are stored locally or in databases for efficient retrieval and reuse.
This architecture ensures smooth communication between system components and supports scalability.

6.3 Dataset Preparation

The dataset used for the system consists of labeled SMS or email messages categorized as spam or ham.
A commonly used dataset is the SMS Spam Collection dataset, which contains thousands of messages labeled accordingly. The dataset is preprocessed before training to remove noise and improve data quality.
Preprocessing steps applied to the dataset include:

- Removing duplicate entries
- Cleaning text by removing punctuation and special characters
- Converting text to lowercase
- Removing stopwords

The dataset is then split into training and testing sets, typically using an 80:20 ratio, to evaluate model performance effectively.

6.4 Model Development

The model development process involves building a pipeline for spam classification.

First, the preprocessed text data is converted into numerical form using TF-IDF vectorization. This step transforms textual data into feature vectors that represent word importance.
Next, machine learning algorithms such as Naïve Bayes, Logistic Regression, or Support Vector Machine are trained on the dataset. Among these, Naïve Bayes is often preferred due to its high efficiency and suitability for text classification tasks.
The trained model learns patterns from the dataset and is evaluated using metrics such as accuracy, precision, recall, and F1-score. The best-performing model is selected and saved for deployment.

6.5 API Development

The backend APIs are developed to enable communication between the frontend and the machine learning model.
Key APIs include:

- Prediction API: Accepts input message and returns classification result (spam/ham)
- Preprocessing API: Handles text cleaning operations
- Model API: Loads and applies the trained model

These APIs are implemented using Flask or Django and follow RESTful principles. They ensure smooth data transfer and real-time prediction.

6.6 Frontend Development

The frontend is designed to provide a simple and user-friendly interface for users.
Users can:

- Enter a message manually
- Submit the message for analysis
- View classification results instantly

The interface displays results clearly, indicating whether the message is spam or safe. Visual indicators such as colors or alerts are used to enhance user experience.

The frontend can be implemented using basic web technologies or advanced frameworks like React for better responsiveness.

6.7 Integration and Deployment

The integration process connects the frontend, backend, and machine learning model into a complete system.

When a user submits a message:

- The frontend sends the request to the backend via API
- The backend preprocesses the message and applies the trained model
- The prediction result is sent back to the frontend
- The result is displayed to the user

The system can be deployed locally or on cloud platforms such as Heroku or AWS. Tools like ngrok can be used for testing APIs locally.

Proper integration ensures efficient performance, real-time prediction, and seamless user interaction.

CHAPTER 7: RESULTS AND ANALYSIS

7.1 Experimental Setup

The Spam Message Detection System was tested using a labeled dataset consisting of SMS or text messages categorized as spam and ham. The dataset contains a balanced mix of legitimate and spam messages to ensure proper evaluation of the model.

The system evaluation focuses on assessing the effectiveness of text preprocessing, feature extraction using TF-IDF, and classification using machine learning algorithms such as Naïve Bayes. The testing process involved providing various input messages through the user interface and observing whether the system correctly classifies them.

The performance of the system was analyzed by comparing predicted outputs with actual labels in the dataset. Additionally, real-time testing was conducted using the developed interface, where users enter messages and receive instant classification results.

7.2 Performance Metrics

Classification Accuracy:
The system achieves high accuracy in distinguishing between spam and ham messages. The trained model correctly classifies the majority of messages, demonstrating its effectiveness in text classification tasks.

Precision:
Precision measures how many messages classified as spam are actually spam. The system shows high precision, meaning it produces very few false spam alerts.

Recall:
Recall indicates how well the system detects actual spam messages. A high recall ensures that most spam messages are successfully identified.

F1-Score:
The F1-score provides a balance between precision and recall, indicating overall model performance. The system maintains a good balance, ensuring reliable predictions.

Overall System Performance:
The system performs efficiently in real-time scenarios, providing fast and accurate predictions with minimal delay.

7.3 Results Explanation

Spam Detection Results:
The system effectively identifies spam messages containing keywords such as “free,” “win,” “offer,” and “prize.” These messages are correctly classified due to the importance assigned by TF-IDF and learned patterns in the model.

Ham Message Classification:
Normal or legitimate messages are accurately classified as ham. The system distinguishes them based on the absence of spam-related patterns.

User Interface Results:
The developed interface allows users to input messages and instantly view results. Spam messages are displayed with warning indicators, while safe messages are marked clearly.

Limitations in Results:
Some borderline cases, such as short messages or ambiguous text, may lead to misclassification. Messages with new or uncommon spam patterns may also affect accuracy.

Figure 6: Home Page Interface

The home page provides the main interface for interacting with the system. It includes options such as single message analysis, batch analysis, and spam keyword viewing.

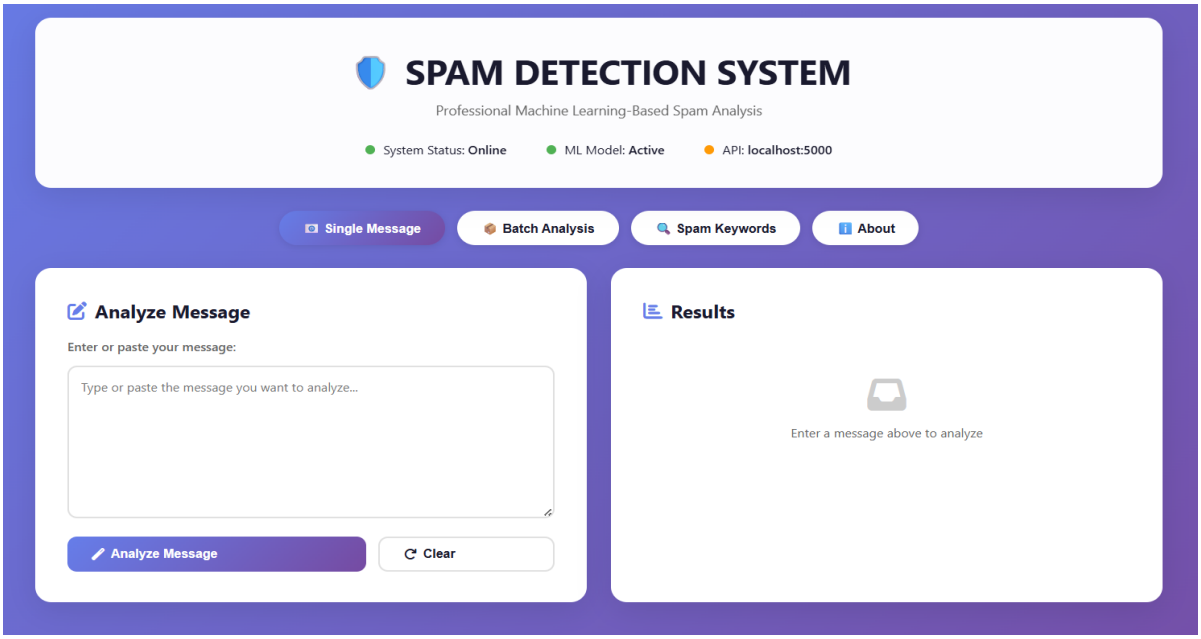


Figure 7: Message Input and Analysis

Users can enter or paste a message into the input box and click the analyze button. The system processes the message and prepares it for classification

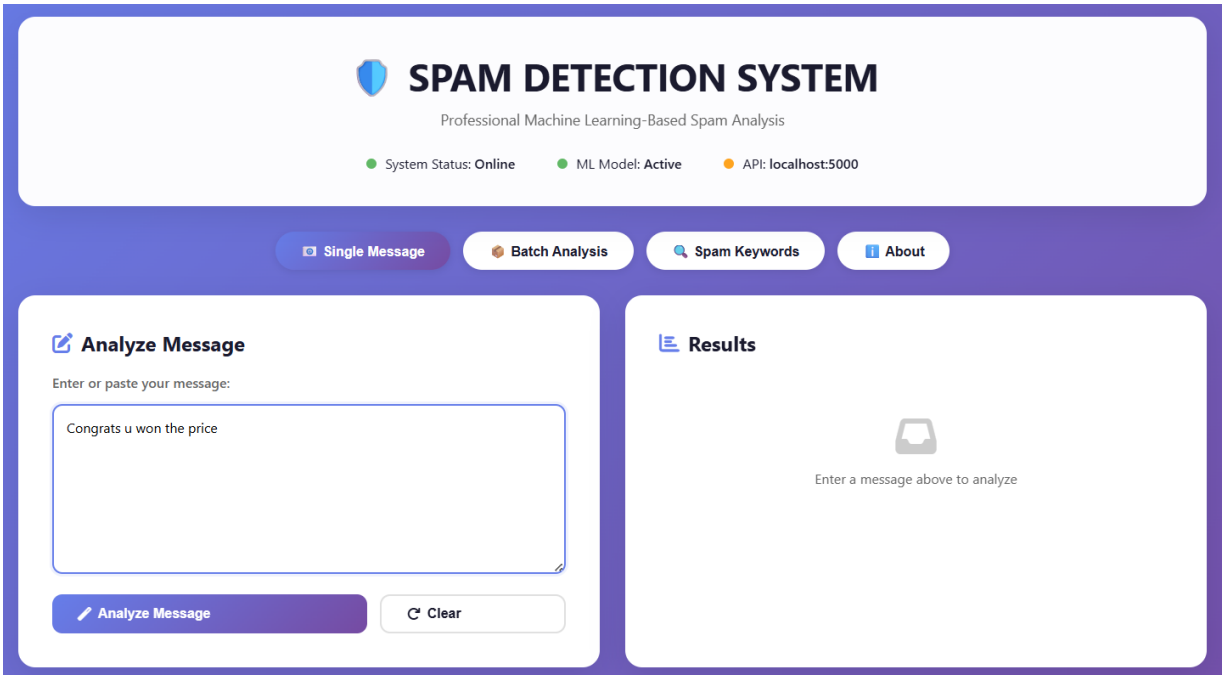
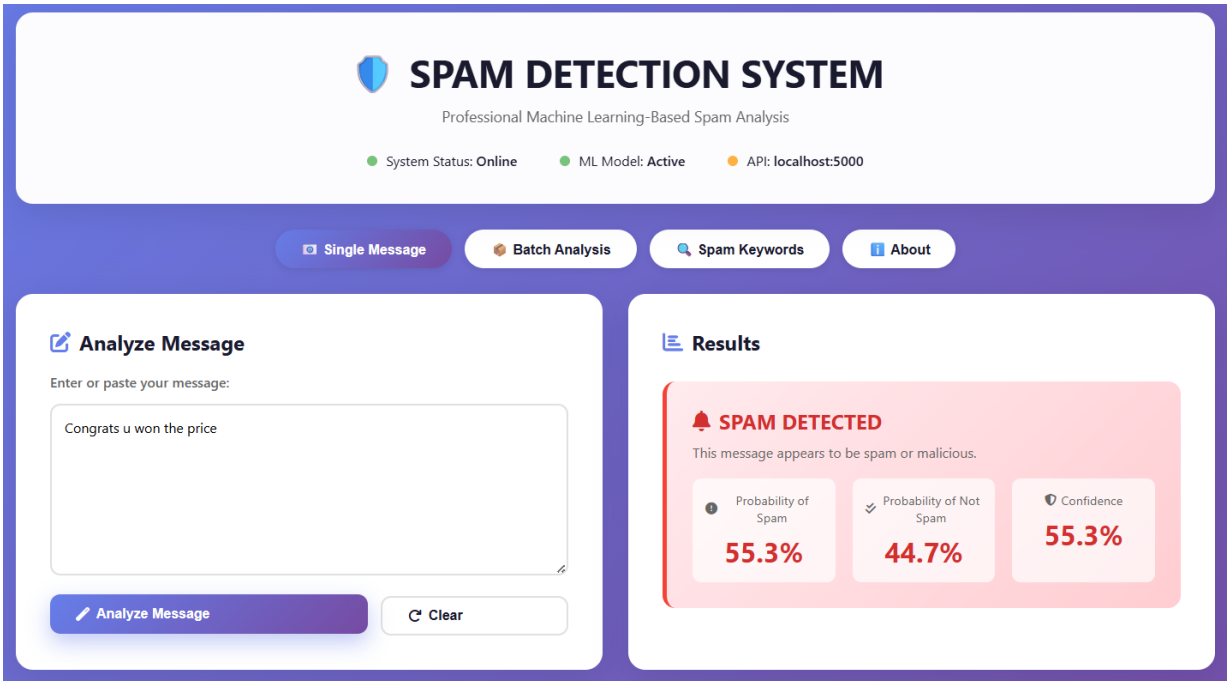


Figure 8: Spam Detection Output

The system displays the result as either spam or ham. Spam messages are highlighted with warning alerts, while safe messages are marked clearly.



7.4 Comparison with Existing Systems

Compared to traditional spam detection methods, the proposed system offers several improvements:

Keyword-Based Systems:
Traditional systems rely only on simple keyword matching, which often results in incorrect classification due to lack of context understanding.

Proposed Spam Detection System:
The proposed system uses TF-IDF and machine learning models to analyze text patterns more effectively, resulting in higher accuracy and better spam detection.

Comparison Table

Feature	Traditional System	Proposed System
Text Processing	Basic	Advanced (NLP)
Feature Extraction	No	Yes (TF-IDF)
Classification	Rule-Based	Machine Learning
Accuracy	Low	High
Real-Time Detection	Limited	Yes
User Interface	Basic	Interactive

7.5 Discussion

The results demonstrate that machine learning-based spam detection significantly improves message classification accuracy compared to traditional methods. The use of TF-IDF vectorization helps in identifying important words and patterns in messages, while the Naïve Bayes classifier efficiently categorizes them. The system is lightweight, fast, and suitable for real-time applications. The developed interface further enhances usability by allowing users to easily analyze messages and view results instantly.

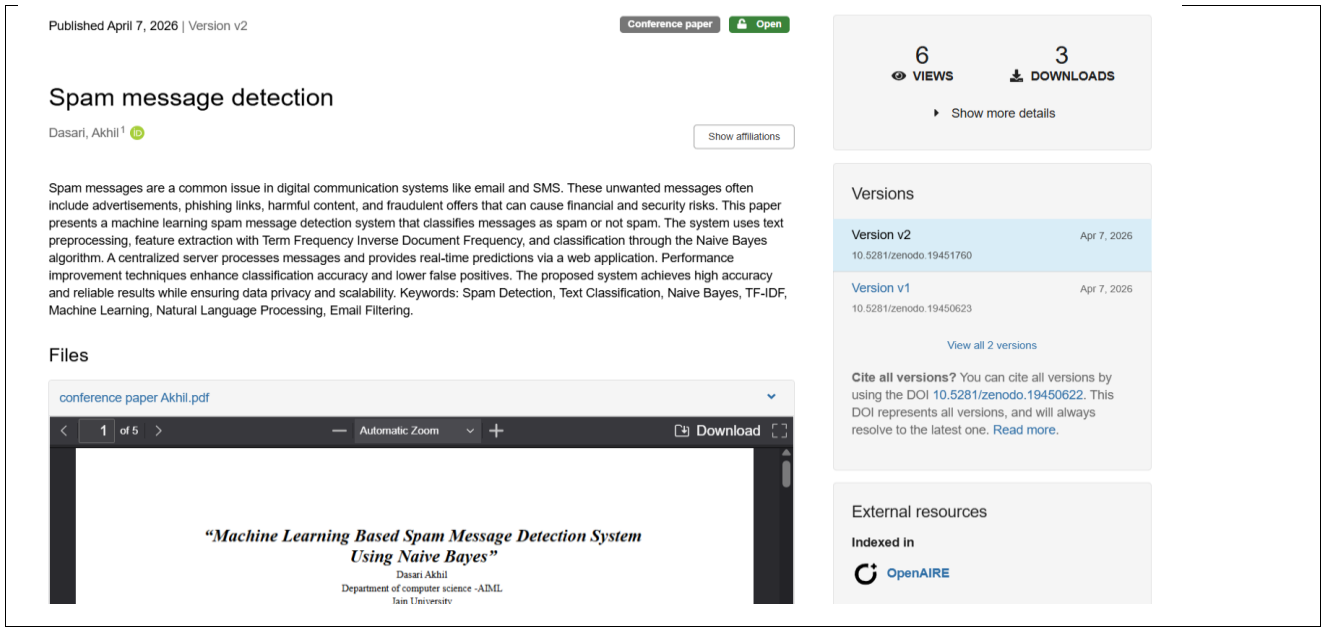
However, there are some limitations. The system may not perform well on highly ambiguous messages or newly emerging spam patterns. It also lacks deep contextual understanding compared to advanced models.

Future improvements may include implementing deep learning models such as LSTM or transformer-based models to enhance accuracy and handle complex text patterns.

CHAPTER 8: PUBLICATIONS

8.1 Published

- **Spam message detection — Zenodo**



Published April 7, 2026 | Version v2

Conference paper | Open

Spam message detection

Dasari, Akhil[1]

Show affiliations

Spam messages are a common issue in digital communication systems like email and SMS. These unwanted messages often include advertisements, phishing links, harmful content, and fraudulent offers that can cause financial and security risks. This paper presents a machine learning spam message detection system that classifies messages as spam or not spam. The system uses text preprocessing, feature extraction with Term Frequency Inverse Document Frequency, and classification through the Naive Bayes algorithm. A centralized server processes messages and provides real-time predictions via a web application. Performance improvement techniques enhance classification accuracy and lower false positives. The proposed system achieves high accuracy and reliable results while ensuring data privacy and scalability. Keywords: Spam Detection, Text Classification, Naive Bayes, TF-IDF, Machine Learning, Natural Language Processing, Email Filtering.

Files

conference paper Akhil.pdf

1 of 5 | Automatic Zoom | Download

"Machine Learning Based Spam Message Detection System Using Naive Bayes"

Dasari Akhil

Department of computer science -AIML

Iain University

6 VIEWS | 3 DOWNLOADS

Show more details

Versions

Version v2 — Apr 7, 2026 — 10.5281/zenodo.19451760

Version v1 — Apr 7, 2026 — 10.5281/zenodo.19450623

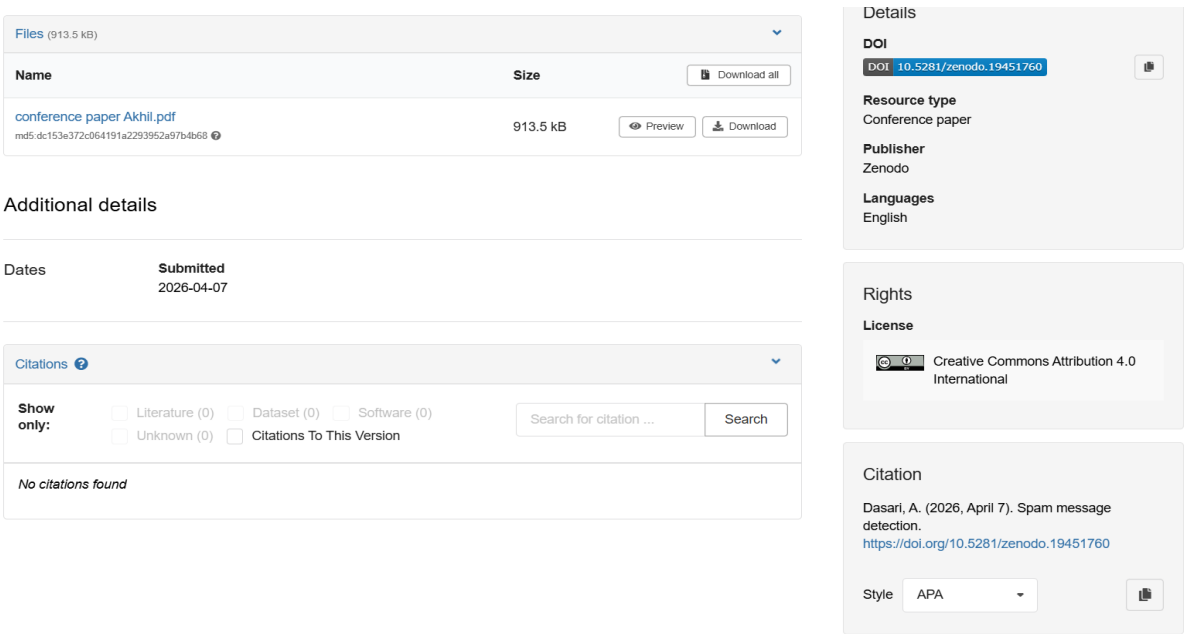
View all 2 versions

Cite all versions? You can cite all versions by using the DOI 10.5281/zenodo.19450622. This DOI represents all versions, and will always resolve to the latest one. Read more.

External resources

Indexed in

OpenAIRE



Files (913.5 kB)

Name	Size	
conference paper Akhil.pdf md5:dc153e372c064191a2293952a97b4b68	913.5 kB	Preview, Download

Download all

Additional details

Dates — **Submitted** 2026-04-07

Citations

Show only: Literature (0), Dataset (0), Software (0), Unknown (0), Citations To This Version

Search for citation ... Search

No citations found

Details

DOI DOI 10.5281/zenodo.19451760

Resource type Conference paper

Publisher Zenodo

Languages English

Rights

License Creative Commons Attribution 4.0 International

Citation

Dasari, A. (2026, April 7). Spam message detection. https://doi.org/10.5281/zenodo.19451760

Style APA

CHAPTER 9: CONCLUSION AND FUTURE WORK

9.1 Conclusion

This project presents the design and implementation of a **Spam Message Detection System**, an intelligent application that uses Natural Language Processing (NLP) and Machine Learning techniques to automatically classify messages as spam or ham. The system addresses the limitations of traditional rule-based filtering methods by introducing data-driven classification using TF-IDF feature extraction and machine learning algorithms.

The implemented system effectively processes raw text messages through preprocessing techniques such as tokenization, stopword removal, and normalization. The use of TF-IDF vectorization enables the system to identify important words and patterns within messages, while the machine learning classifier, particularly Naïve Bayes, efficiently predicts whether a message is spam or legitimate.

The system is developed as a user-friendly application with an interactive interface that allows users to input messages and receive real-time classification results. It reduces manual effort in identifying unwanted messages, improves filtering accuracy, and enhances communication safety.

Overall, the project demonstrates that machine learning-based approaches can significantly improve spam detection performance by providing fast, scalable, and reliable classification solutions.

9.2 Achievements

The project successfully achieved the following objectives:

- Developed a functional spam detection system integrating frontend, backend, and machine learning components

- Implemented text preprocessing techniques for cleaning and normalizing message data
- Applied TF-IDF vectorization for effective feature extraction
- Built and trained a machine learning model (Naïve Bayes) for spam classification
- Designed a real-time prediction system for message analysis
- Developed a user-friendly interface for input and output visualization
- Achieved high accuracy in classifying spam and ham messages
- Demonstrated improved performance compared to traditional keyword-based filtering methods

9.3 Limitations

Despite its effectiveness, the system has certain limitations:

- The system relies on training data and may not perform well on completely new or unseen spam patterns
- It does not fully capture deep contextual meaning of messages compared to advanced NLP models
- Short or ambiguous messages may sometimes be misclassified
- The model may require retraining to adapt to evolving spam techniques
- It does not currently detect malicious links or attachments within messages
- Multilingual message support is limited

9.4 Future Enhancements

Several improvements can be made to enhance the system further:

- **Advanced Deep Learning Models:**
  Implement models such as LSTM and transformer-based models like BERT to improve contextual understanding and classification accuracy

- **Real-Time Spam Filtering Integration:**
  Integrate the system with email services or messaging platforms for automatic spam filtering
- **Phishing and URL Detection:**
  Extend the system to identify malicious links, phishing attempts, and fraudulent content within messages
- **Multilingual Support:**
  Enhance the system to support multiple languages using multilingual NLP models
- **Continuous Learning System:**
  Implement feedback-based learning where the model improves over time using user feedback
- **Spam Keyword Updating:**
  Dynamically update spam keywords and patterns based on new trends
- **Batch Processing and Analytics:**
  Improve batch message analysis and provide insights such as spam statistics and trends
- **Mobile Application Development:**
  Develop a mobile version of the system for wider accessibility
- **Explainable AI:**
  Provide explanations for predictions (e.g., highlighting words that contributed to spam classification)

REFERENCES

[1] [1] Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media, Inc.

[2] [2] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.

[3] [3] Sparck Jones, K. (1972). *A Statistical Interpretation of Term Specificity and Its Application in Retrieval*. Journal of Documentation, 28(1), 11–21.

[4] [4] McCallum, A., & Nigam, K. (1998). *A Comparison of Event Models for Naïve Bayes Text Classification*. AAAI Workshop.

[5] [5] Joachims, T. (1998). *Text Categorization with Support Vector Machines: Learning with Many Relevant Features*. European Conference on Machine Learning (ECML).

[6] [6] Rennie, J. D. M., Shih, L., Teevan, J., & Karger, D. R. (2003). *Tackling the Poor Assumptions of Naïve Bayes Text Classifiers*. ICML.

[7] [7] Androutsopoulos, I., Koutsias, J., Chandrinos, K. V., & Spyropoulos, C. D. (2000). *An Experimental Comparison of Naïve Bayesian and Keyword-Based Anti-Spam Filtering*. SIGIR Workshop.

[8] [8] Almeida, T. A., Gómez Hidalgo, J. M., & Yamakami, A. (2011). *Contributions to the Study of SMS Spam Filtering*. ACM Symposium on Document Engineering.

[9] [9] Hidalgo, J. M. G. (2002). *Evaluating Cost-Sensitive Unsolicited Bulk Email Categorization*. SAC Conference.

[10] [10] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. NAACL-HLT.

[11] [11] Hochreiter, S., & Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation, 9(8), 1735–1780.

[12] [12] Vaswani, A., et al. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems (NeurIPS).

[13] [13] Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). *Efficient Estimation of Word Representations in Vector Space*. arXiv preprint.
[14] [14] Pennington, J., Socher, R., & Manning, C. D. (2014). *GloVe: Global Vectors for Word Representation*. EMNLP.
[15] [15] Sebastiani, F. (2002). *Machine Learning in Automated Text Categorization*. ACM Computing Surveys, 34(1), 1–47.